

Create authentication service that returns JWT

Code - >

The screenshot shows an IDE with the SpringLearnApplication project open. The left sidebar displays the project structure, including src/main/java/com/cognizant/spring_learn, config, controller, model, resources, and test. The main editor area shows the JwUtil.java file, which contains the following code:

```

package com.cognizant.spring_learn.util;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

public class JwUtil {

    // Must be at least 32 characters (256 bits)
    private static final String SECRET = "ThisIsASecretKeyForJWTGeneration123456";

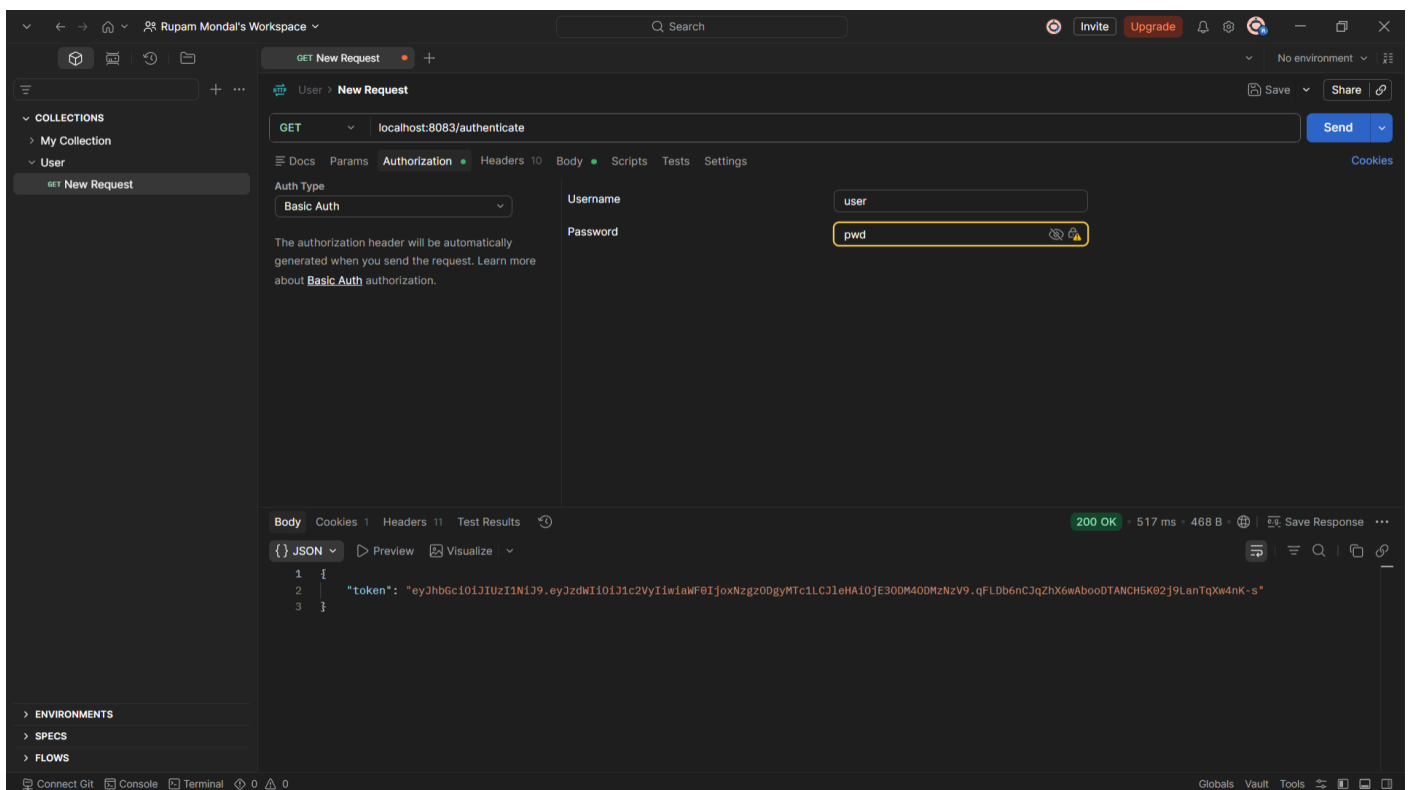
    private static final SecretKey KEY = Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));

    public static String generateToken(String username) {

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 20 * 60 * 1000))
            .signWith(KEY, SignatureAlgorithm.HS256)
            .compact();
    }
}

```

Output(Postman)

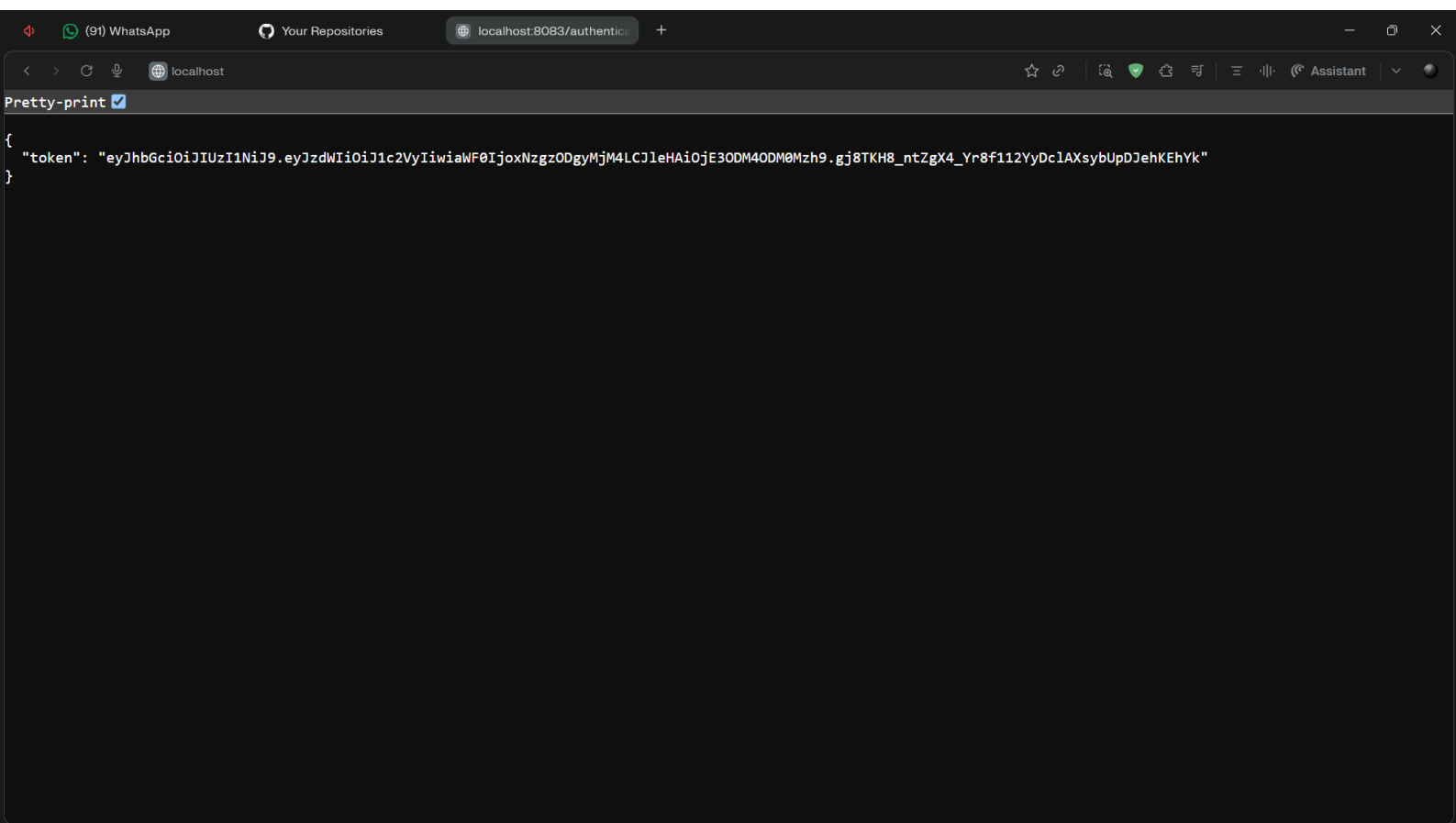


f

Output (Chrome)

Username - user

password - pwd



The screenshot shows a web browser window with the address bar displaying 'localhost:8083/authentic...'. The browser's developer tools are open, and the 'Pretty-print' tab is selected in the console. The console displays a JSON object with a single key-value pair: 'token' mapped to a long alphanumeric string. The browser's tab bar shows 'Your Repositories' and '(91) WhatsApp'.

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzg3ODgyMjM4LCJleHAiOjE3ODM4ODM0Mzh9.gj8TKH8_ntZgX4_Yr8f112YyDclAXsybUpDJehKEhYk"  
}
```